MediQ

MediQ Project
Medical Queue & Reservation System

INDEX

01 Overview

02 Architecture & Design

03 Key Implementation

04 Technical Decisions

05 TROUBLE SHOOTING

06 Retrospective & Improvement

Skills

FRONTEND

선언적 렌더링을 통한
직관적인 인터페이스 구축

BACKEND

엔터프라이즈급 안정성을
갖춘 백엔드 로직 구현

DATABASE

정규화된 관계형 데이터베이스
설계 및 최적화

INFRASTRUCTURE

안정적이고 가용한
클라우드 인프라 아키텍처

DEVOPS

CI/CD 파이프라인 자동화 및
컨테이너 오케스트레이션

기획 배경

끝이 보이지 않는 대기 시간, 단절된 의료 경험

의료 현장의 Pain Point

병원 (Hospital)

- 과도한 **전화 예약 응대 부하**로 인한 업무 마비
- **무분별한 노쇼(No-Show)** 발생으로 인한 병원 수익 및 시간 손실

환자 (Patient)

- 아픈 몸을 이끌고 견뎌야 하는 **기약 없는 대기 시간**
- 진료 후 처방전을 들고 **약국을 다시 찾아 헤매야 하는 번거로움**

MediQ 핵심 솔루션

실시간 알림

실시간 대기열

대기 인원이 줄어들 때마다 현재 남은 순번을 웹 푸시(Web Push)로 알려주어, 무의미한 현장 대기를 없앴습니다.

동선 최적화

스마트 동선

병원 검색 결과에서 주변 약국 탐색으로 즉시 전환하여 치료의 흐름을 하나로 이어갑니다.

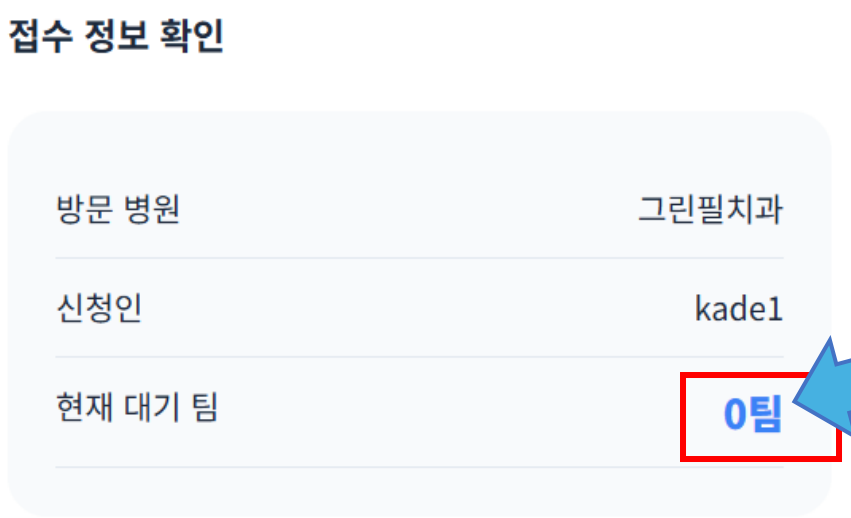
노쇼 방지

안전 예약

예약금 결제 시스템으로 노쇼를 효과적으로 방지하고, 상황에 맞는 합리적인 환불 로직을 구현합니다.

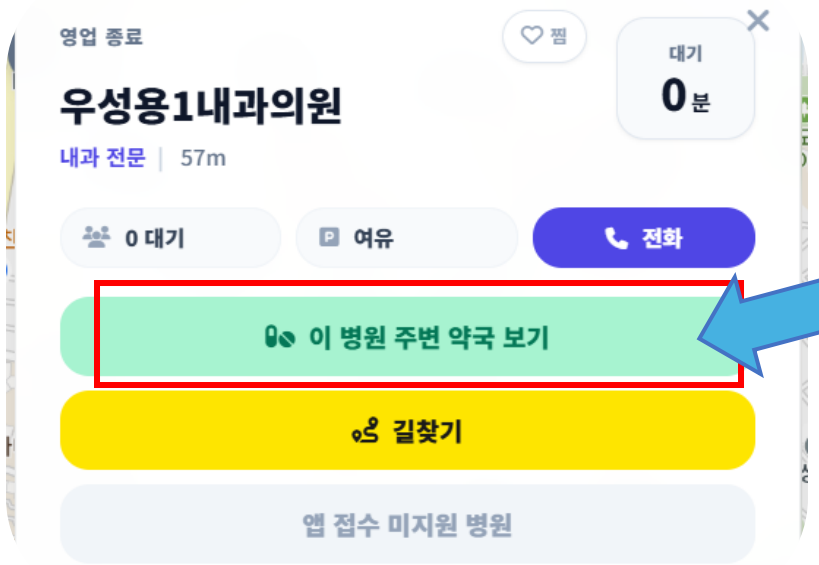
핵심 차별점 : 동선의 연속성

환자가 병원 문을 나서는 순간, 다음 목적지인 약국까지의 동선을 이미 손 안에 쥐여줍니다.



AS-IS : 단절된 경험

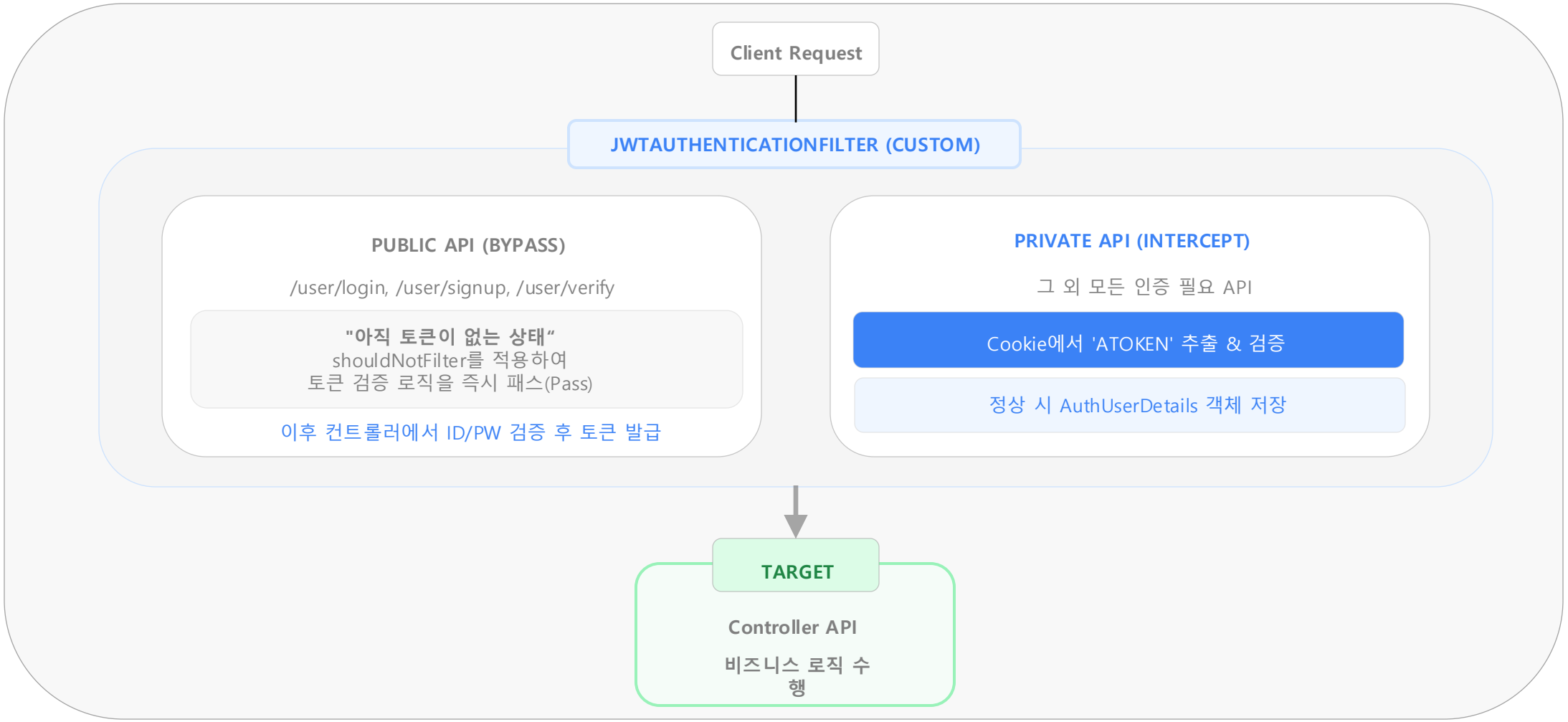
혼잡한 대기실에서의 기약 없는 기다림과
진료 후 약국을 다시 찾아 헤매는 피로감



TO-BE : 동선의 연속성

실시간 내 순서 확인부터 맞춤형
주변 약국 스마트 매칭까지 끊김 없는 연결

01 Custom JWT Filter Architecture



어디서 어떻게 적용되는가?

클라이언트의 요청이 서블릿(DispatcherServlet)을 거쳐 컨트롤러에 도달하기 전 동작합니다. OncePerRequestFilter를 상속받아 한 번의 요청당 한 번만 검사하며, 헤더가 아닌 **쿠키(Cookie)에서 'ATOKEN'을 추출**합니다. 검증 성공 시 커스텀 AuthUserDetails 객체를 생성해 SecurityContext에 등록합니다.

로그인할 때 필터를 안 거치는 이유?

로그인이나 회원가입 시점에는 사용자가 **제출할 토큰이 아직 없는 상태**입니다. 따라서 shouldNotFilter 메서드를 오버라이딩하여 해당 경로(/user/login 등)는 **토큰 검사 없이 필터를 패스(Bypass)**시킵니다. 이후 Controller 단에서 아이디/비밀번호를 검증하고 새로운 JWT 토큰을 발급하도록 역할을 명확히 분리했습니다.

02 OAuth 2.0 Social Login Process

OAuth2AuthorizationRequestRepository

01. 요청 & 상태 유지
인가 과정 중 필요한 임시 데이터를 세션이 아닌 **AES256 암호화 쿠키**에 저장하여 무상태(Stateless) 유지

→

CustomOAuth2UserService

02. 정보 획득 & 식별
DB 조회 후 최초 가입자라면 임의의 **UUID 비밀번호**를 생성하여 강제 회원가입 처리

→

OAuth2SuccessHandler

03. 자체 JWT 발급
소셜 토큰 대신 자체 제작한 JWT를 **HttpOnly 쿠키(ATOKEN)**로 프론트엔드에 안전하게 전달

완벽한 STATELESS 아키텍처 구현
소셜 로그인 진행 시 세션(Session)을 사용할 경우 서버 확장성(Scale-out)에 문제가 생깁니다. 이를 방지하기 위해 OAuth2AuthorizationRequestRepository를 직접 구현하여 상태 유지에 필요한 임시 데이터를 **AES256으로 암호화된 쿠키**에 저장함으로써 완벽한 무상태성을 달성했습니다.

XSS 방어 및 소셜 종속성 탈피
소셜 플랫폼의 토큰을 그대로 쓰지 않고 OAuth2SuccessHandler를 통해 권한(Role)이 담긴 자체 JWT를 재발급했습니다. 특히 토큰을 로컬 스토리지에 저장하지 않고 **HttpOnly 속성이 부여된 쿠키**로 발급하여 자바스크립트를 통한 XSS 해킹 공격을 원천 차단했습니다

로그인

서비스 이용을 위해 로그인해주세요.

일반 환자 | 병원 관계자

이메일

비밀번호

로그인

카카오로 로그인

회원가입 | 아이디 찾기 | 비밀번호 찾기

로그인

서비스 이용을 위해 로그인해주세요.

일반 환자 | 병원 관계자

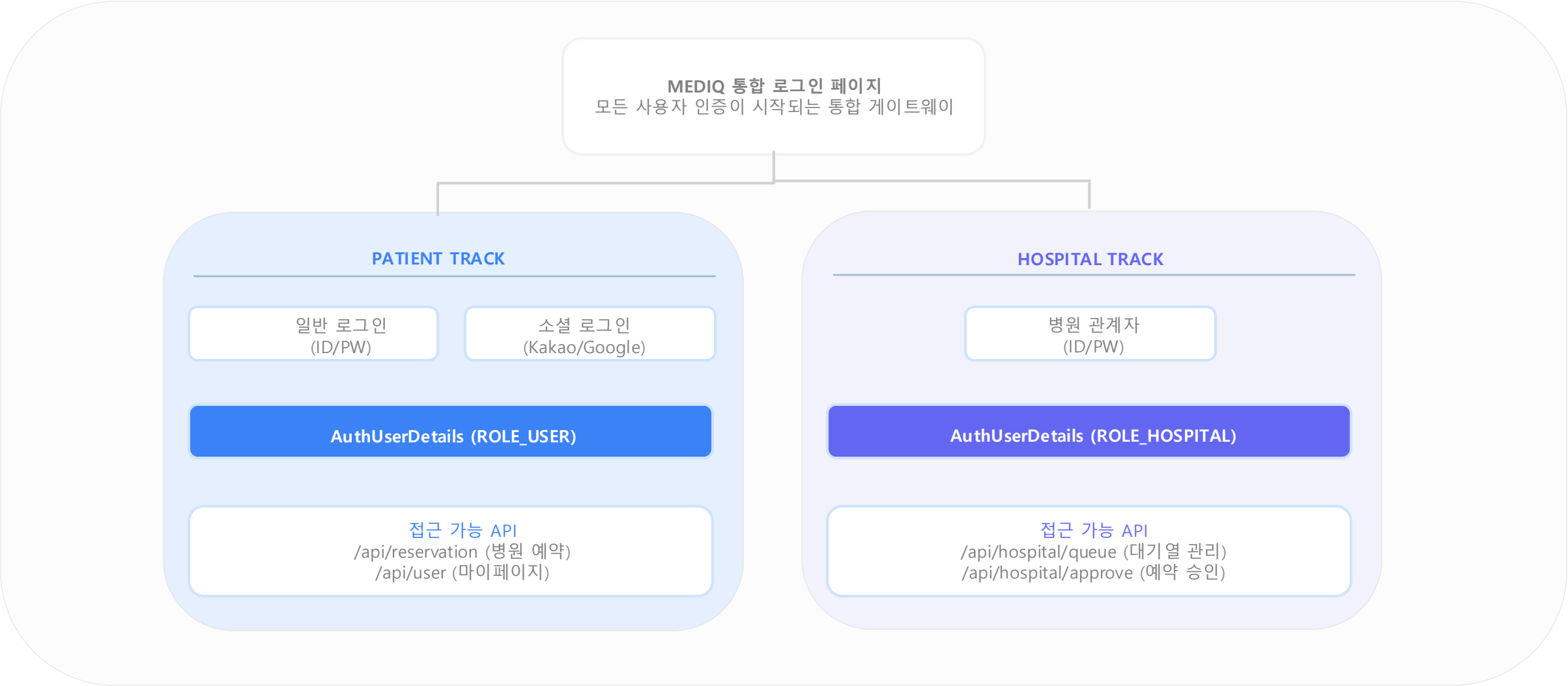
병원 담당자 이메일

비밀번호

로그인

회원가입 | 아이디 찾기 | 비밀번호 찾기

03 Role-Based Access Control (권한 분리 전략)



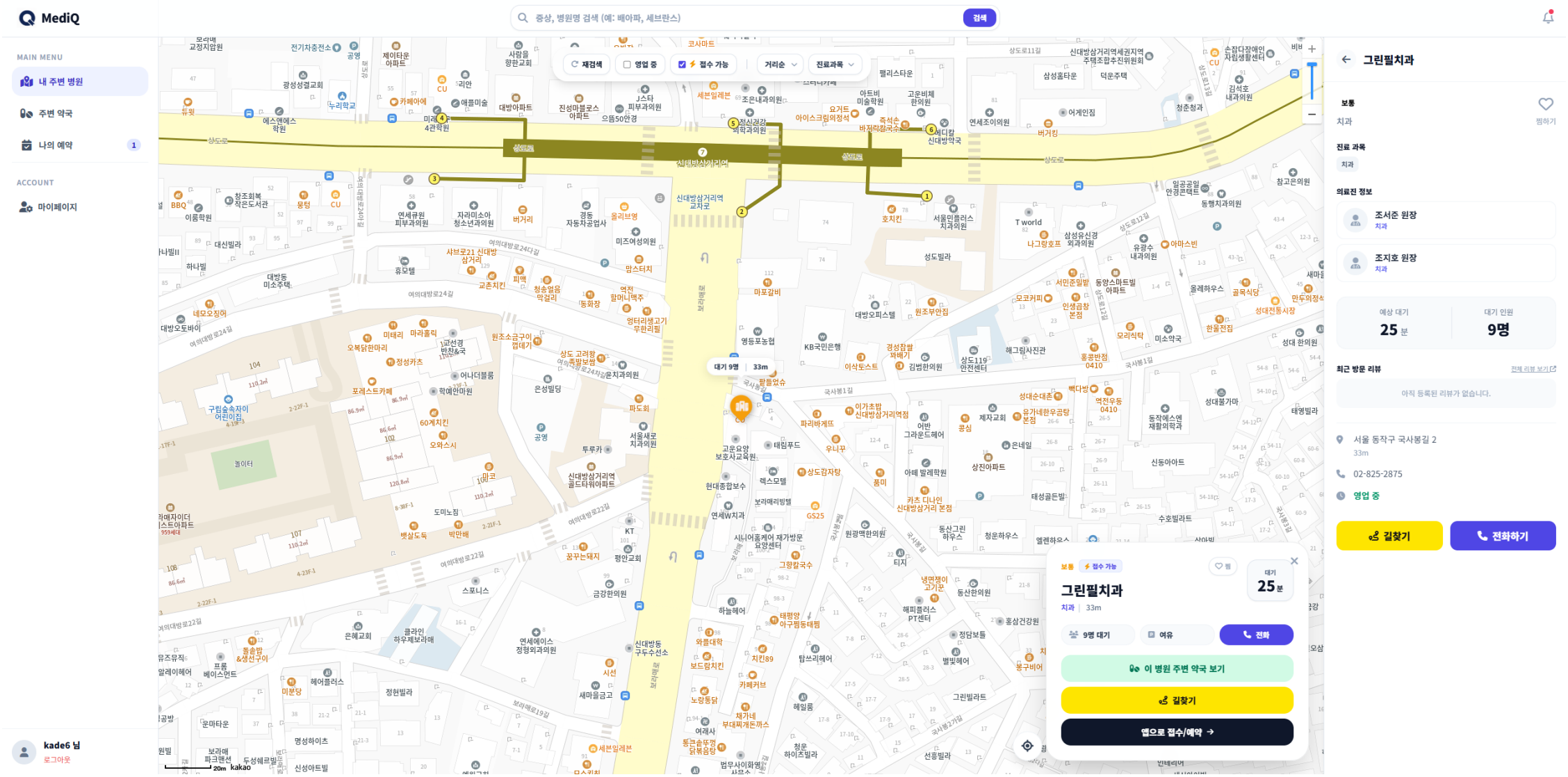
통합 게이트웨이 및 권한 분기
모든 사용자는 하나의 **통합 로그인 페이지**에서 인증을 시작합니다. 일반 사용자는 ID/PW 또는 소셜 로그인을 통해 ROLE_USER를 부여받고, 병원 관계자는 병원 전용 계정으로 로그인하여 ROLE_HOSPITAL을 부여받도록 동적 분기 처리를 구현했습니다.

인가 (AUTHORIZATION) 제어
로그인 수단에 상관없이, 발급된 토큰 내부의 Role을 기반으로 Spring Security가 **API 엔드포인트를 철저히 격리**합니다. 만약 일반 환자의 토큰으로 병원 관리 API에 접근하려 할 경우 시큐리티 필터단에서 즉시 차단(403)하여 시스템 안전을 완벽히 보장합니다.

설계 포인트 및 기대 효과

공통 응답 포맷 (BASERESPONSE)
모든 API의 응답을 {success, code, message, result} 형태의 일관된 JSON 규격으로 통일했습니다. 2000번대(성공), 3000번대(입력 오류) 등 자체 Enum 상태 코드를 정의하여 프론트엔드의 파싱 에러를 방지하고, Axios 인터셉터 등을 통한 에러 처리 로직을 중앙화할 수 있도록 협업 생산성을 극대화했습니다.

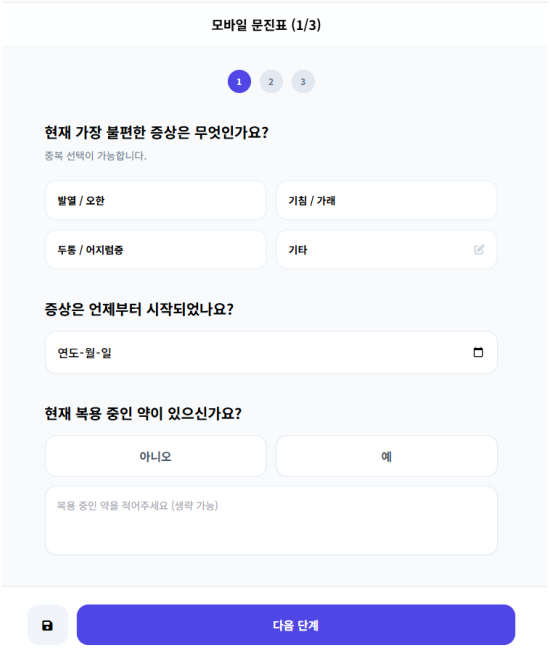
AOP 기반 예외 처리 (CLEAN CODE)
@RestControllerAdvice를 활용하여 예외 처리를 전역으로 분리했습니다. 컨트롤러나 서비스 계층에서 지저분한 try-catch 블록을 제거하고 핵심 비즈니스 로직에만 집중할 수 있게 되었으며, 예측 불가능한 스택 트레이스(Stack Trace) 노출을 막아 보안성을 크게 높였습니다.

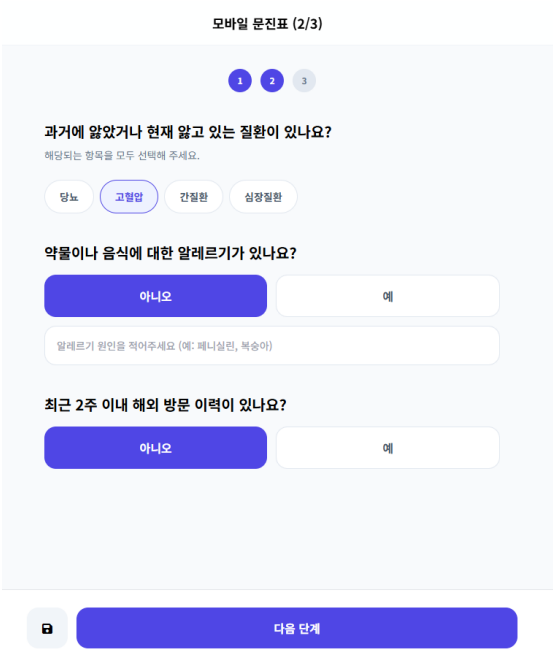

MediQ
증상, 병원명 검색 (예: 배아파, 세브란스)
검색
MAIN MENU
내 주변 병원
주변 약국
나의 예약
ACCOUNT
마이페이지
kade6 님
로그아웃
재검색
영업 중
접수 가능
거리순
진료과목
그린필치과
보통
치과
찜하기
진료 과목
의료진 정보
조서준 원장
조지호 원장
예상 대기
25 분
대기 인원
9명
최근 방문 리뷰
아직 등록된 리뷰가 없습니다.
서울 동작구 국사봉길 2
33m
02-825-2875
영업 중
길찾기
전화하기
보통 접수 가능
그린필치과
치과 | 33m
대기
25 분
9명 대기
여유
전화
이 병원 주변 약국 보기
길찾기
앱으로 접수/예약 →

1

사전 문진표 작성 (SURVEYFORMSERVICE)

예약 진행 전, 환자의 증상, 기저질환, 알레르기 정보를 입력받습니다. @NotNull 어노테이션으로 필수 값을 1차 차단하며, 통과된 데이터는 현재 로그인 유저 (AuthUserDetails)와 매핑되어 DB에 안전하게 저장됩니다.

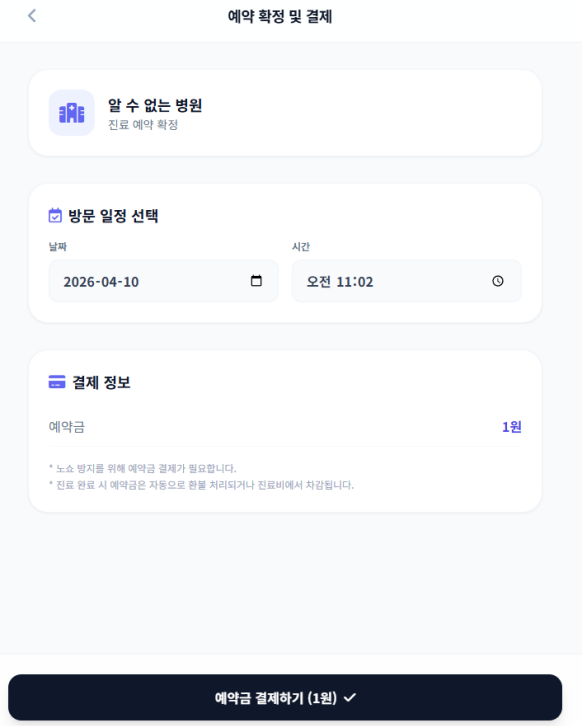




2

주문서 선(先) 생성 (ORDERSSERVICE.CREATE)

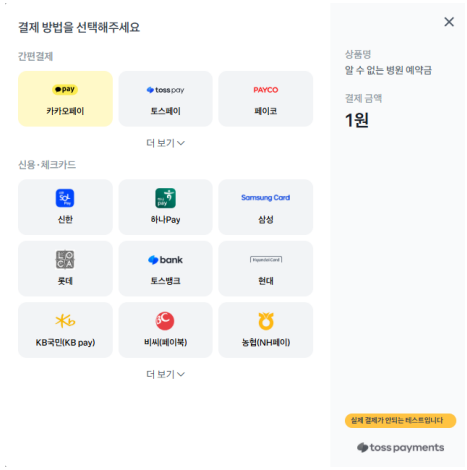
포트원(PG) 결제창을 띄우기 직전, 서버는 DB에서 해당 병원의 정확한 예약금을 조회하여 **결제 대기(paid = false) 상태의 Orders 객체를 DB에 먼저 생성**합니다. 이는 클라이언트가 결제 금액을 임의로 결정하지 못하게 하는 기준점이 됩니다.

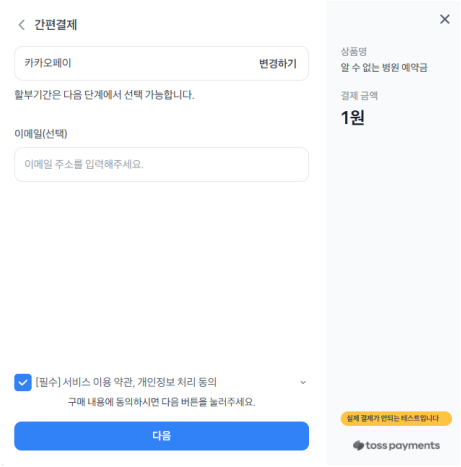


3

포트원 결제 및 서버 교차 검증 (ORDERSSERVICE.VERIFY)

프론트엔드에서 전송한 결제 신호를 믿지 않고, 서버가 PaymentClient를 통해 **포트원 서버와 직접 비동기 통신**을 수행하여 실제 결제 금액을 교차 검증하는 'Zero Trust' 설계입니다.



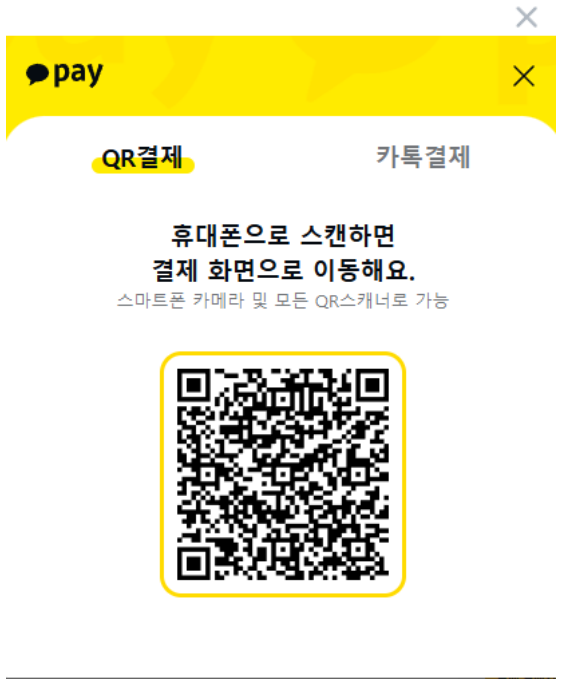


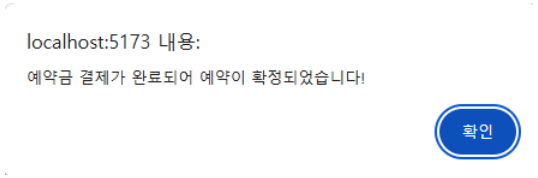

```java
// 포트원 결제 완료 후 금액 검증
@Transactional  1개 사용 위치
public void verify(AuthUserDetails user, OrdersDto.VerifyReq dto) {
    CompletableFuture<Payment> completableFuture = pg.getPayment(dto.getPaymentId());
    Payment payment = completableFuture.join();

    if (payment instanceof PaidPayment paidPayment) {
        Map<String, Object> customData = new GsonBuilder()
                .setObjectToNumberStrategy(ToNumberPolicy.LONG_OR_DOUBLE)
                .create().fromJson(paidPayment.getCustomData(), Map.class);

        Long ordersIdx = Long.parseLong(customData.get("ordersIdx").toString());
        Orders orders = ordersRepository.findById(ordersIdx)
                .orElseThrow(() -> new RuntimeException("주문 정보를 찾을 수 없습니다."));

        // 우리 DB에 저장된 원래 내야 할 예약금 금액
        int totalPrice = orders.getPaymentPrice();

        // 유저가 실제로 결제한 금액과 우리 DB의 금액이 일치하면 결제 완료 처리!
        if (paidPayment.getAmount().getTotal() == totalPrice) {
            orders.setPaid(true);
            orders.setPgPaymentId(dto.getPaymentId());
            ordersRepository.save(orders);

            PaymentHistory paymentHistory = PaymentHistory.builder()
                    .user(orders.getUser()) // 결제한 유저
                    .paymentName(orders.getHospital().getName() + " 예약금")
                    .amount((long) totalPrice) // 결제한 금액
                    .paymentMethod("앱결제") // 결제 수단
                    .paymentStatus("결제완료") // 현재 상태
                    .reservationId(orders.getIdx()) // 어떤 예약에 대한 결제인지 연결고리 (주문번호 저장)
                    .build();

            paymentHistoryRepository.save(paymentHistory); // DB에 저장

        }
    }
}
```

4

예약 확정 & 결제 내역 기록

교차 검증을 통과하면 Orders 엔티티의 pgPaymentId를 업데이트하고, 동시에 PaymentHistory 엔티티를 생성하여 DB에 영구 기록합니다. 하나의 @Transactional 안에서 처리되어 데이터 파편화를 방지합니다.

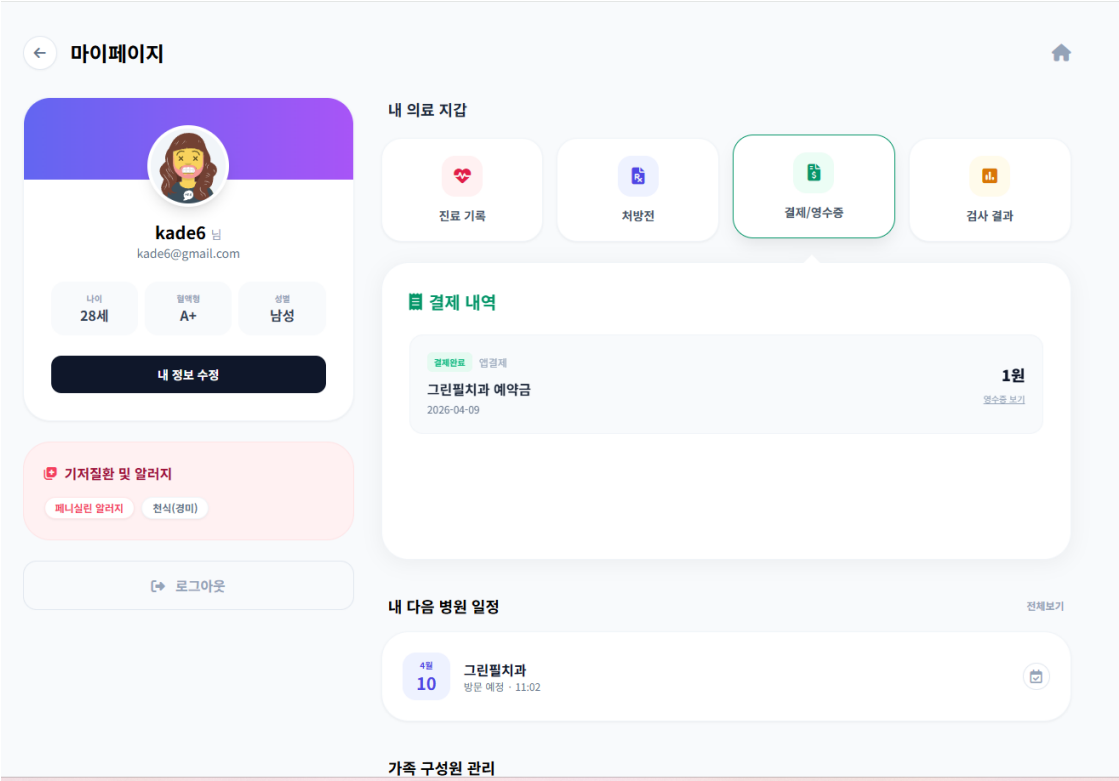




5

마이페이지 데이터 연동 (GETMYSCHEDULE)

Java Stream API의 filter(order -> order.isPaid())를 활용하여 검증이 완료된 예약만 추출합니다. 프론트엔드에서 바로 사용할 수 있도록 월/일 형식으로 정제하고 UI 클래스까지 맵핑하여 반환합니다.



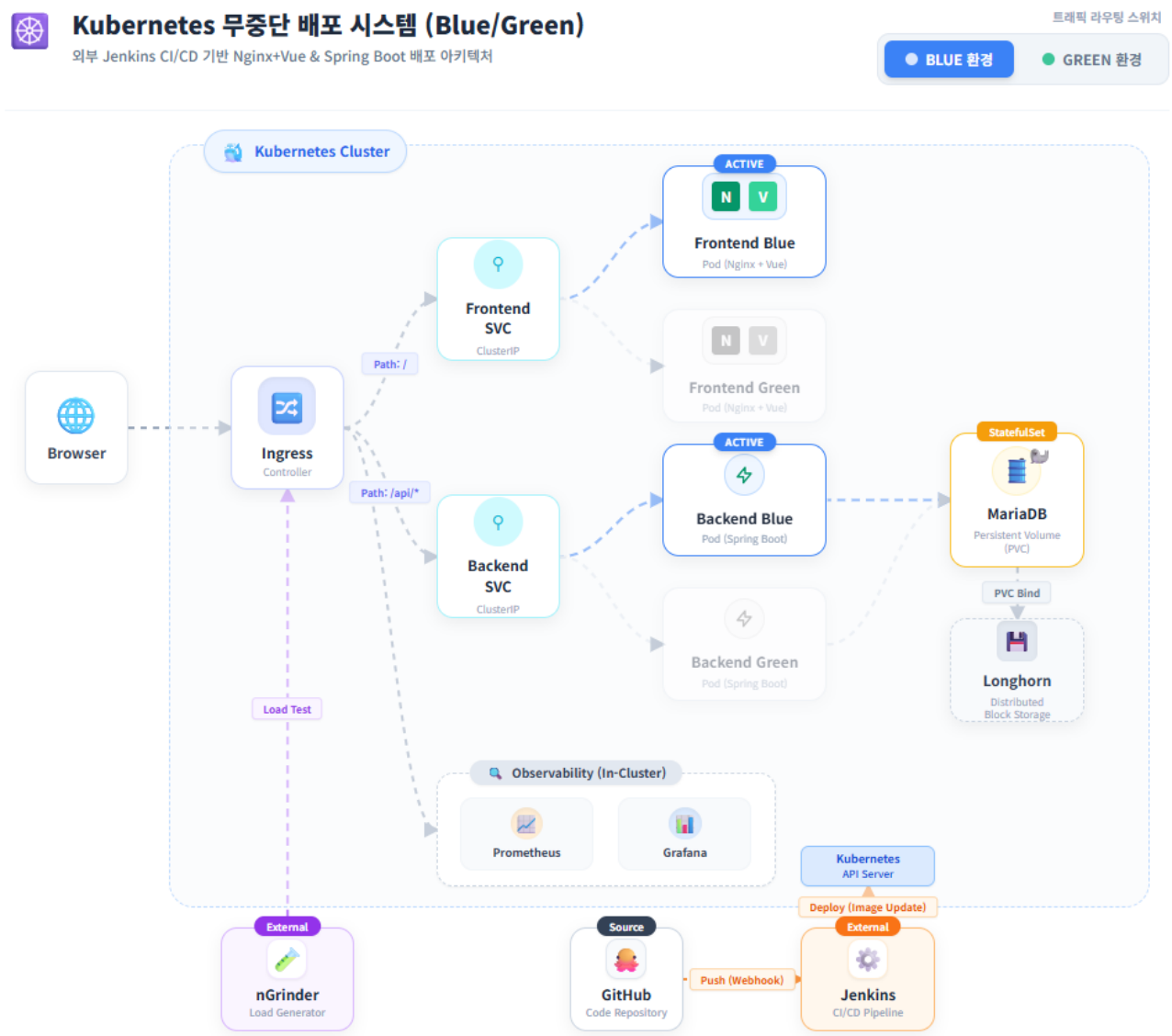
금액 위변조(FORGERY) 원천 차단

클라이언트에서 전송한 "결제 완료" 신호만 믿고 DB를 조작할 경우, 해커가 브라우저 개발자 도구를 통해 금액 변수를 100원으로 조작하는 심각한 재무적 해킹에 노출됩니다. **Zero Trust 아키텍처 기반의 서버 측 PG사 직접 교차 검증**은 이러한 해킹을 원천 차단합니다.

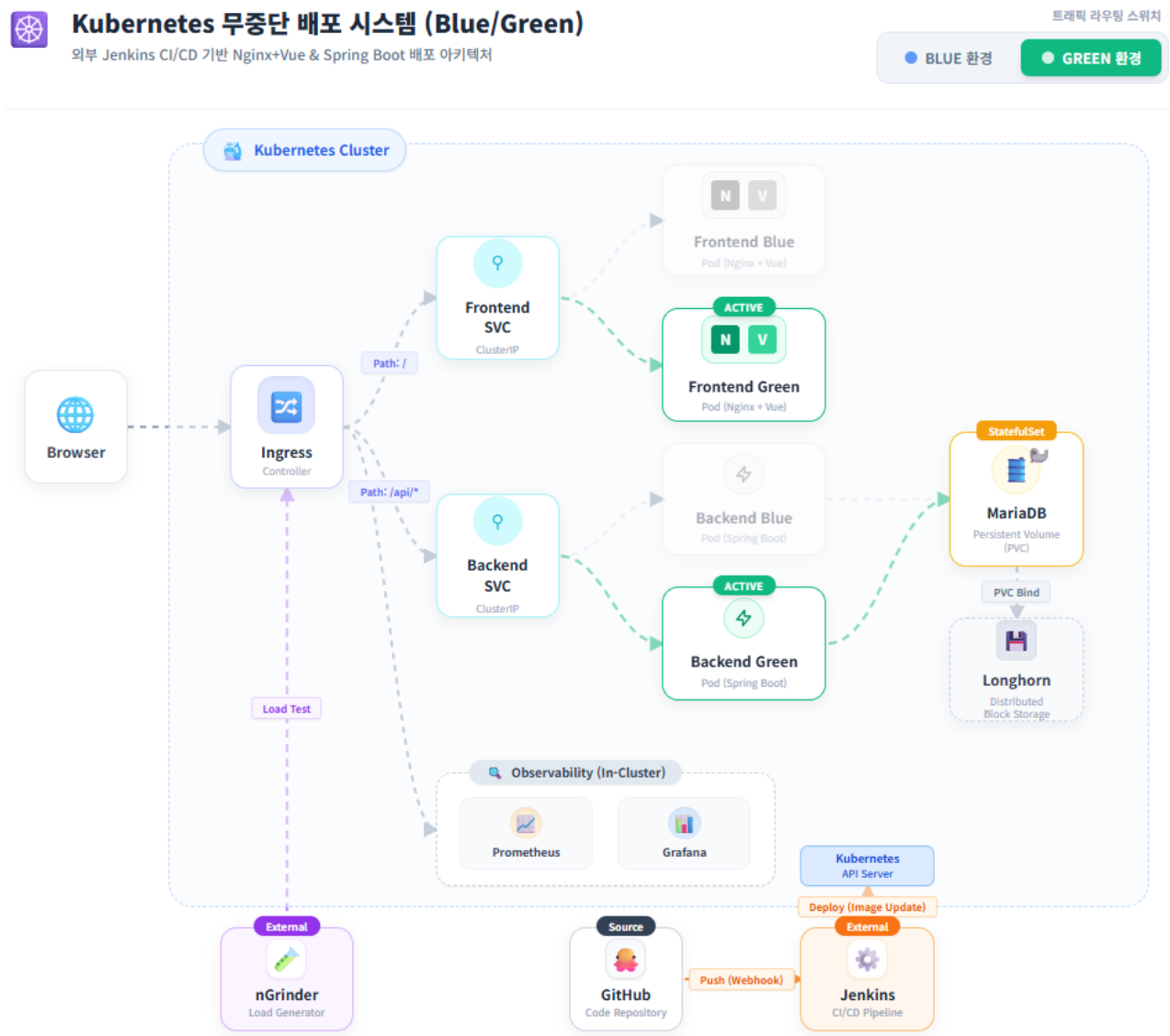
고스트 결제 방어 및 확장성 확보

PG사 결제는 성공했는데 서버 DB 반영에 실패하는 '데이터 불일치(고스트 결제)'를 막기 위해, **주문서를 먼저 생성(Step 2)한 뒤 검증하는 방식**을 채택했습니다. 또한, 저장된 pgPaymentId를 통해 노쇼(No-show) 발생 시 즉각적인 서버 환불 API 통신이 가능한 유연한 구조를 완성했습니다.

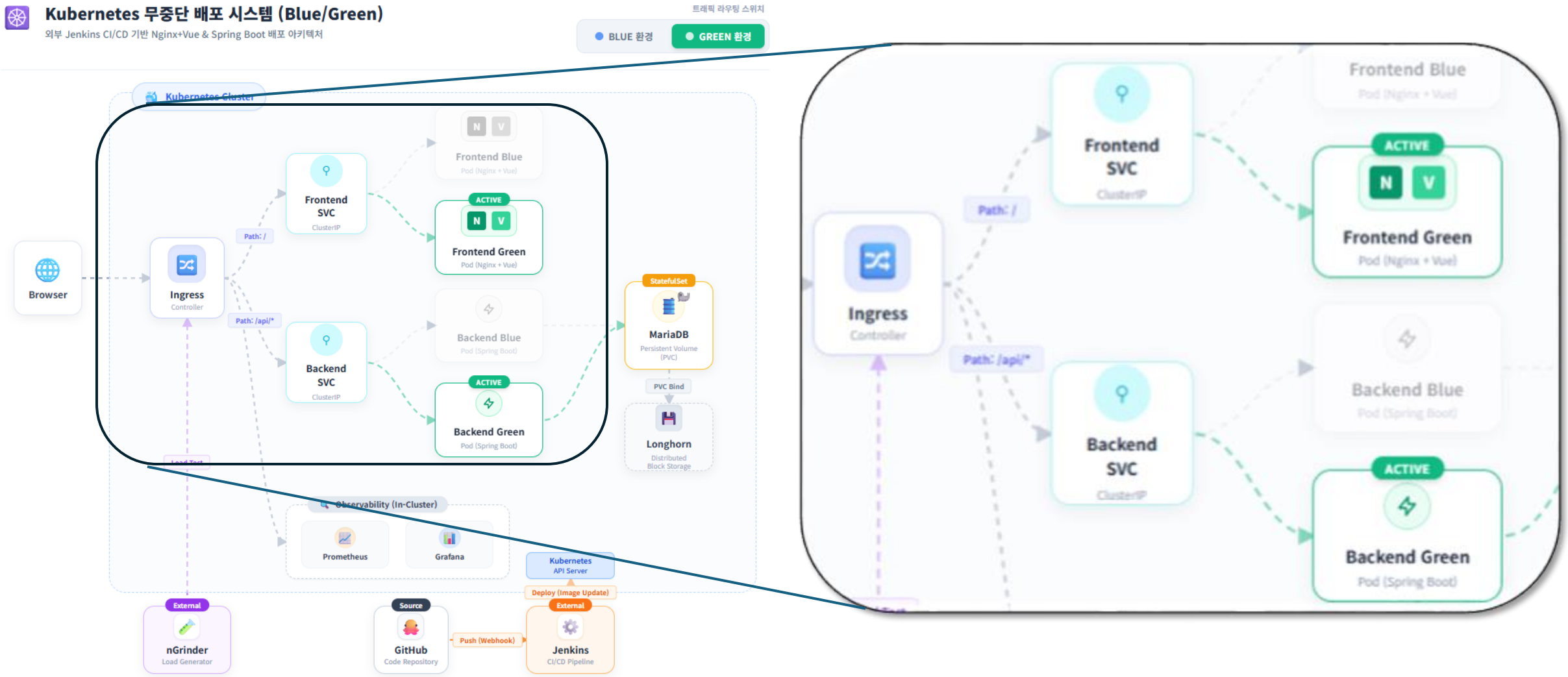
Architecture & Design



Architecture & Design

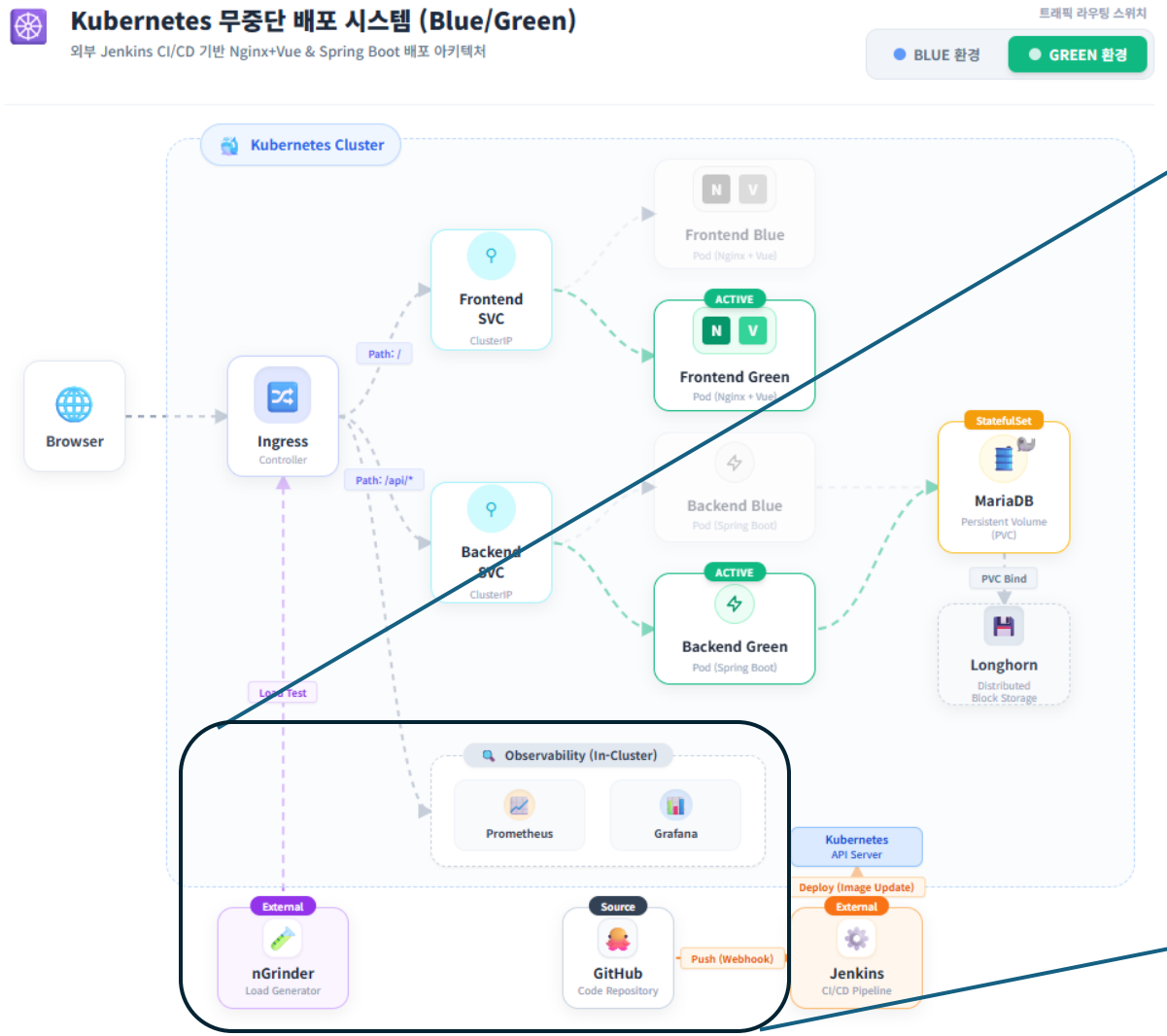


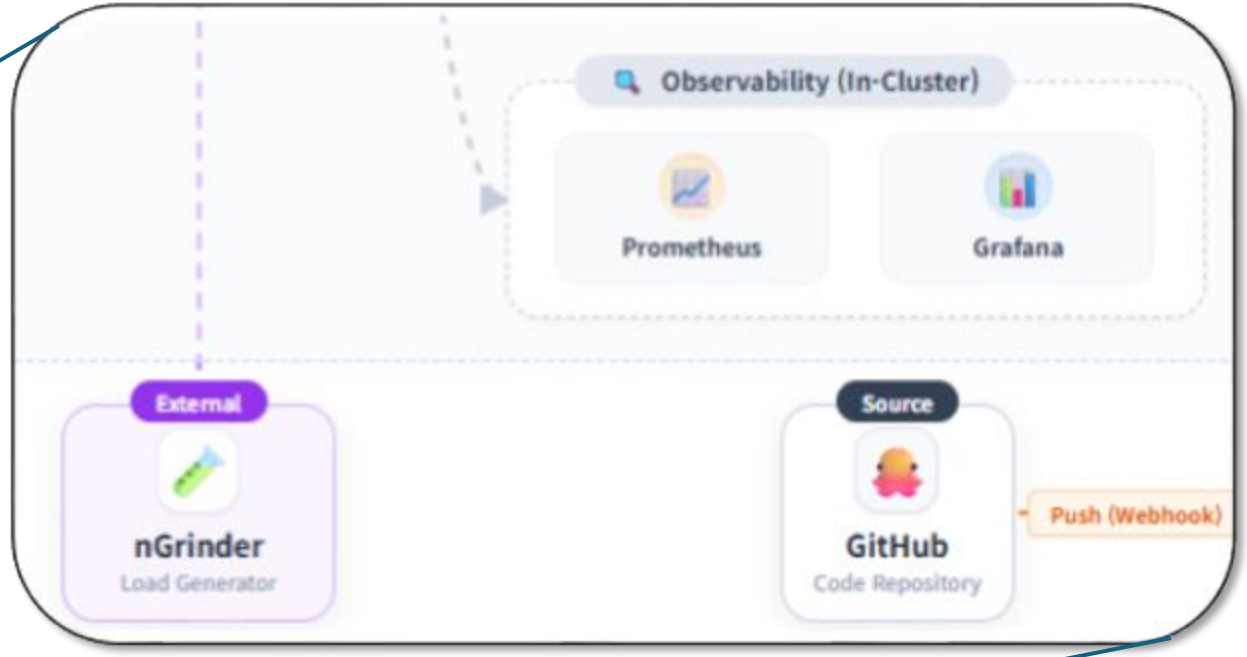
Architecture & Design



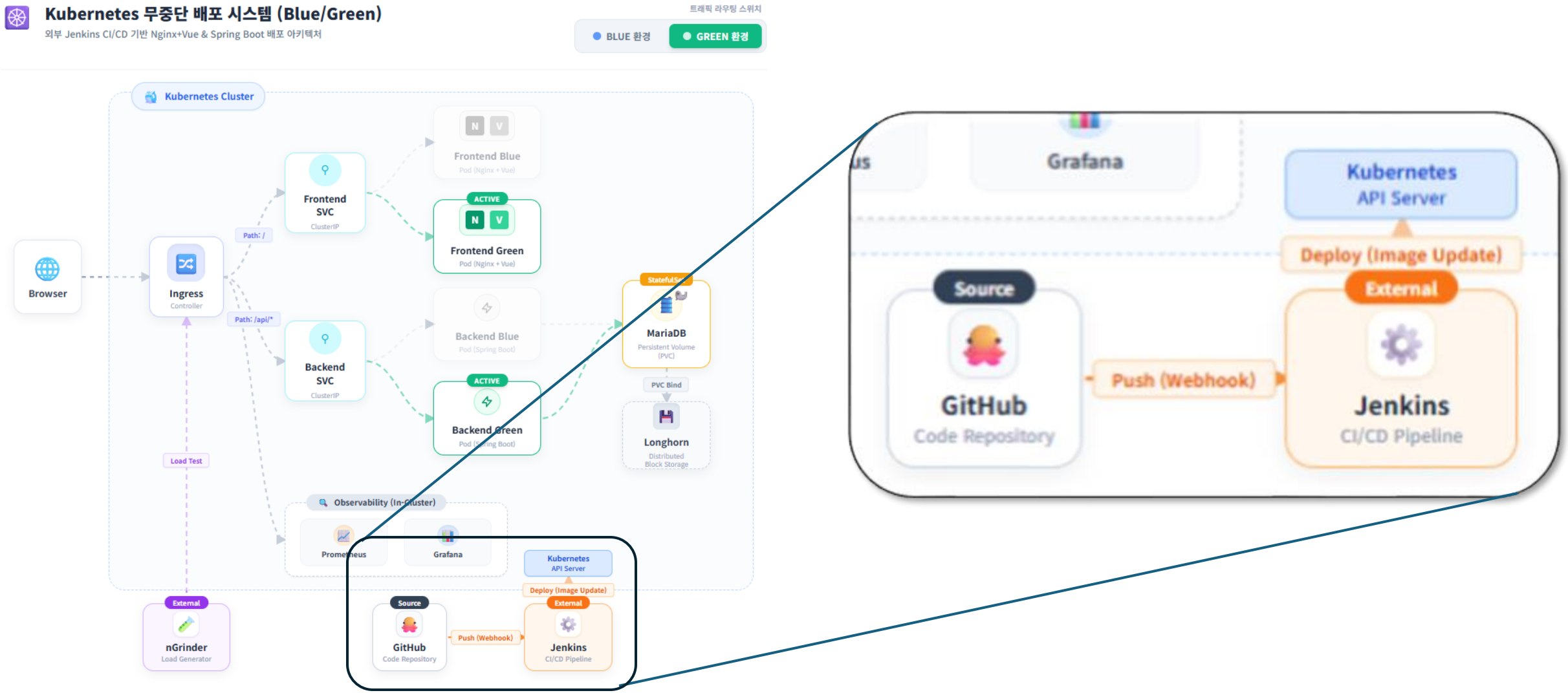
Architecture & Design

Architecture & Design





Architecture & Design



가족 구성원 추가 기능

- 로그인한 사용자의 마이페이지에서 가족 구성원 추가

가족 구성원 추가 기능

raw JSON Schema Beautify

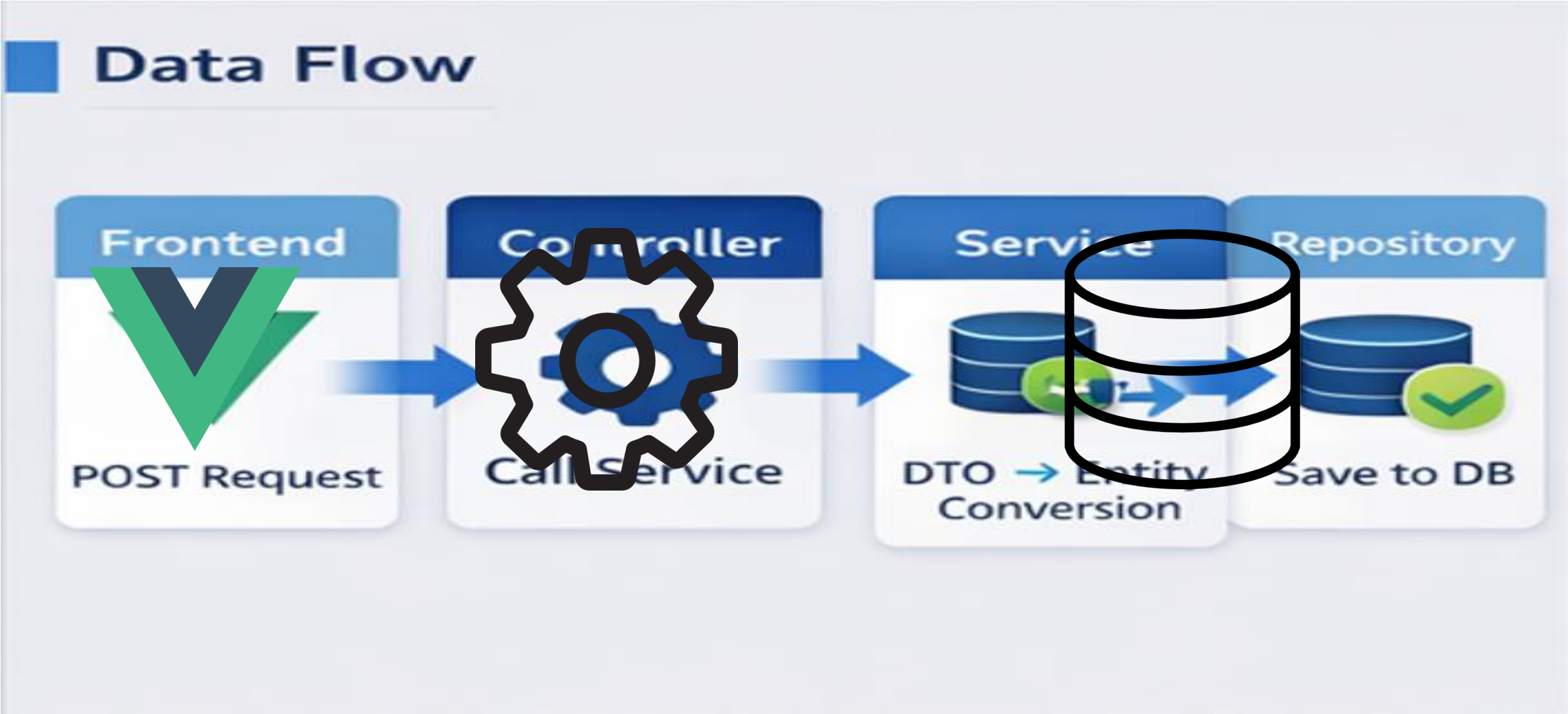
```
1 {
2     "name": "이항별",
3     "age": 25,
4     "bloodtype": "O형",
5     "relation": "아들"
6 }
```

Body 200 OK 576 ms 499 B Save Response

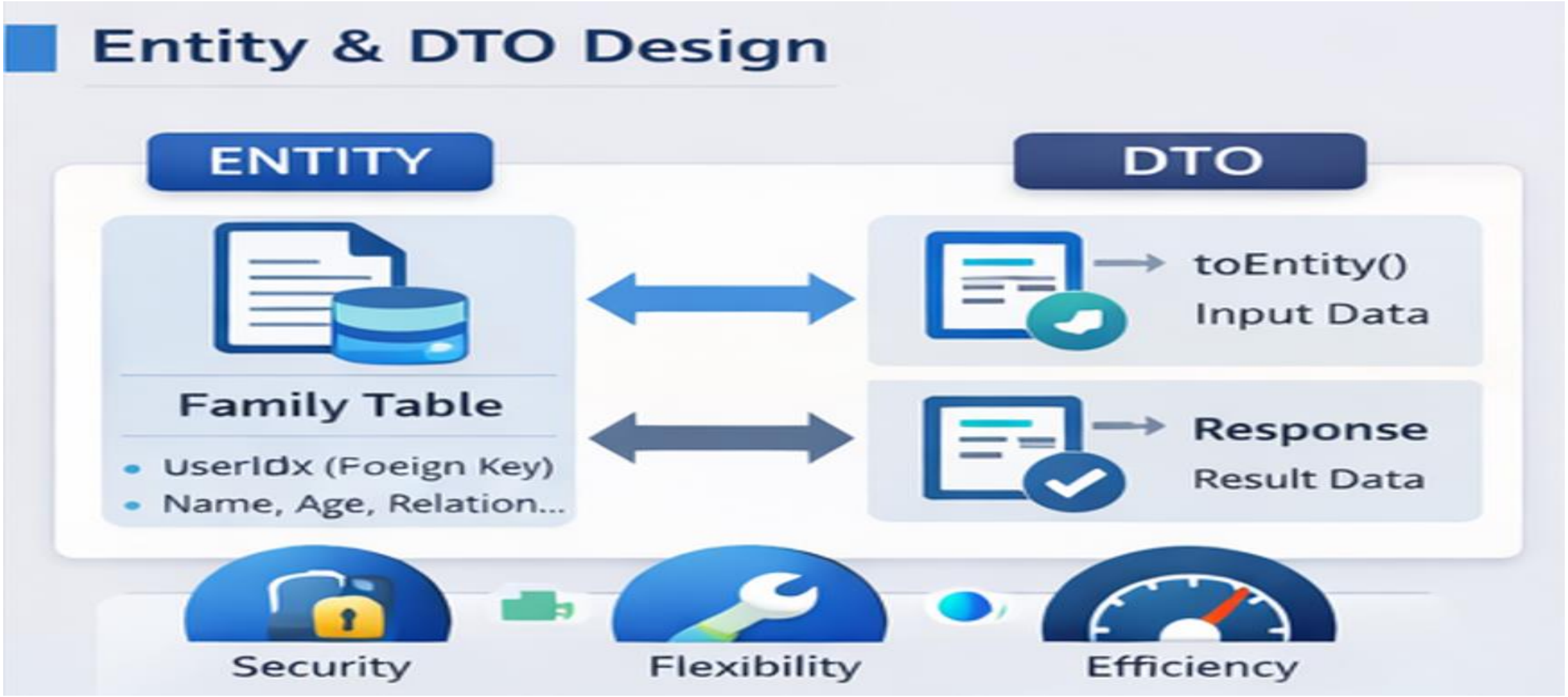
{ } JSON Preview Visualize

```
1 {
2     "idx": 1,
3     "name": "이항별",
4     "age": 25,
5     "bloodtype": "O형",
6     "relation": "아들"
7 }
```

가족 구성원 추가 기능


Data Flow
Frontend
POST Request
Controller
Call Service
Service
DTO → Entity
Conversion
Repository
Save to DB

가족 구성원 추가 기능


Entity & DTO Design
ENTITY
DTO
Family Table
UserIdx (Foeign Key)
Name, Age, Relation...
toEntity()
Input Data
Response
Result Data
Security
Flexibility
Efficiency

리뷰 작성 기능

raw JSON Schema Beautify

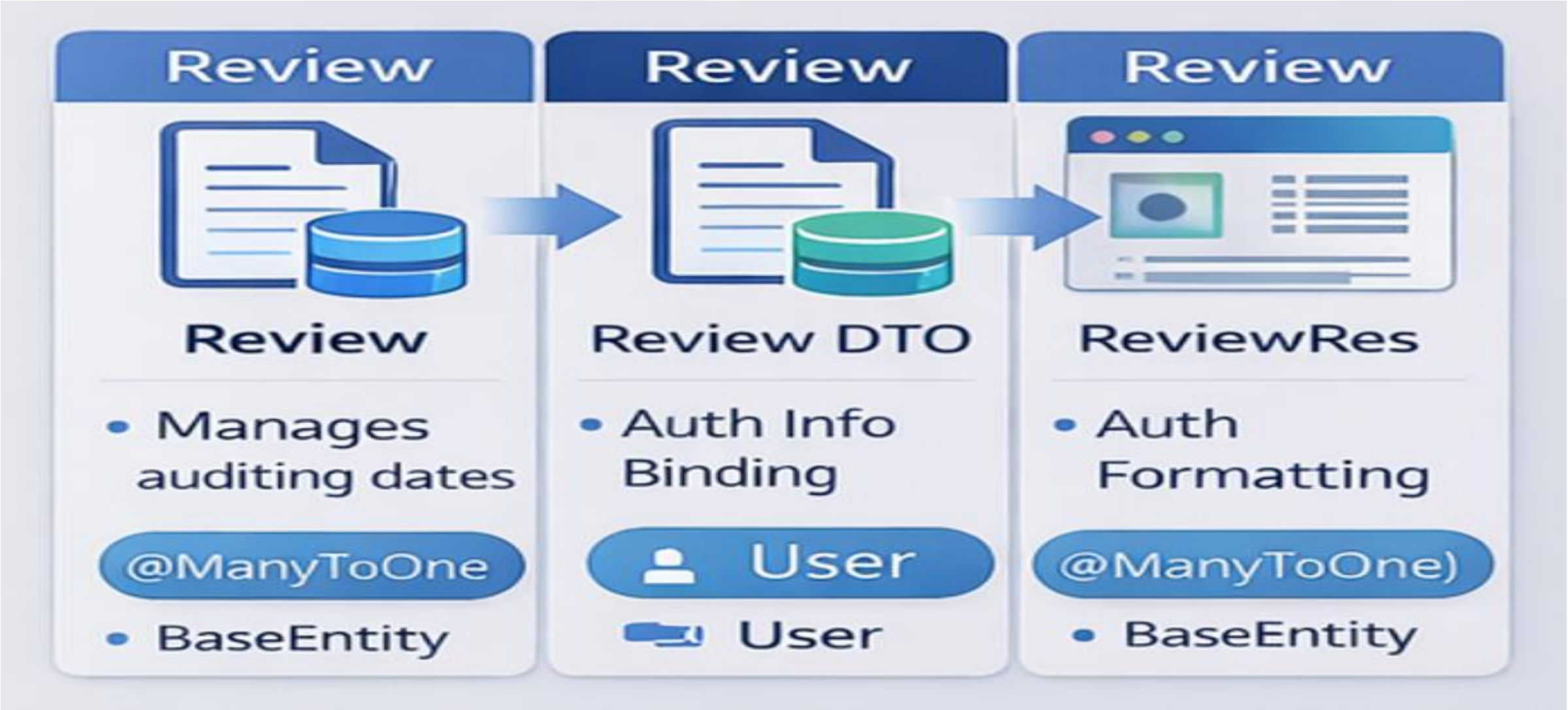
```
1 {
2     "contents": "건물의 청결해요"
3 }
```

Body 200 OK 153 ms 568 B Save Response

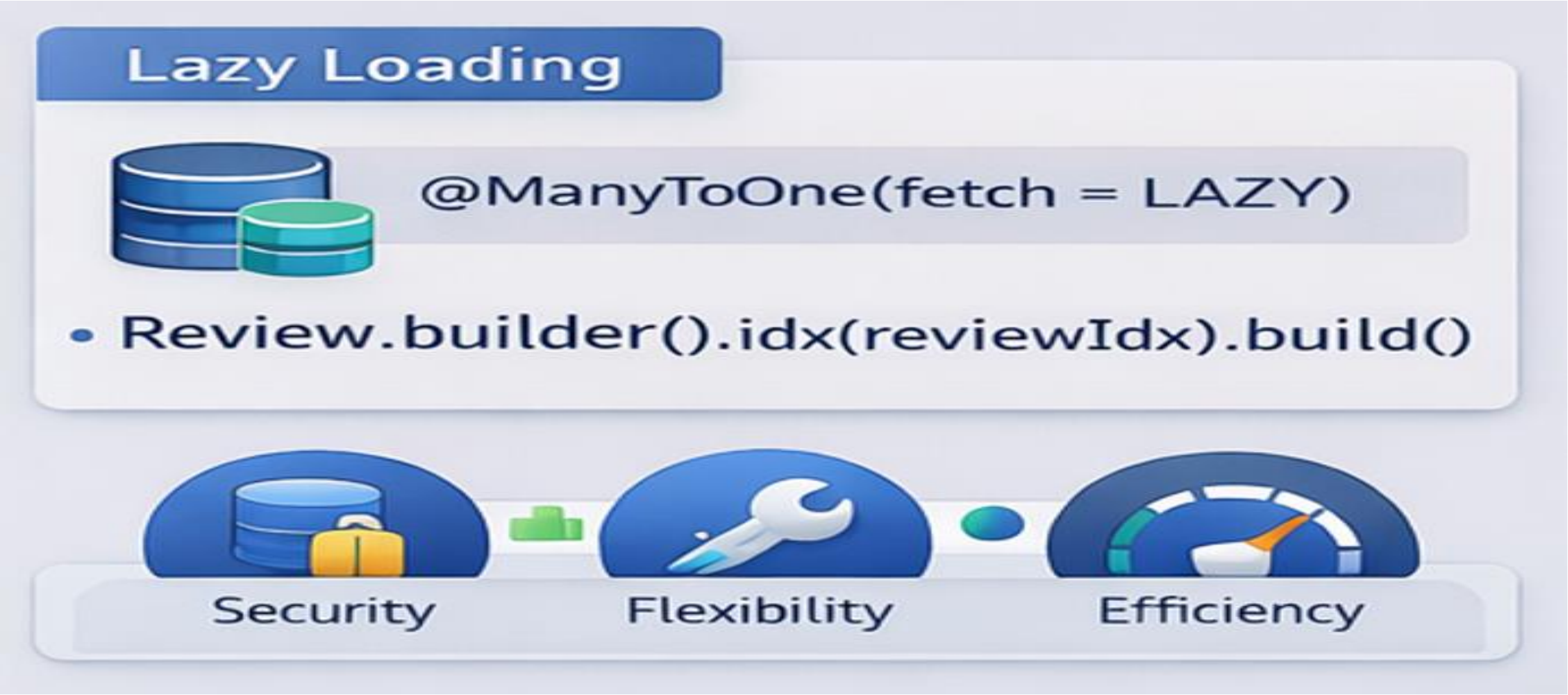
{} JSON Preview Visualize

```
1 {
2     "success": true,
3     "code": 2000,
4     "message": "요청이 성공했습니다",
5     "result": {
6         "idx": 1,
7         "contents": "건물의 청결해요",
8         "writer": "노응안"
```

리뷰 작성 기능

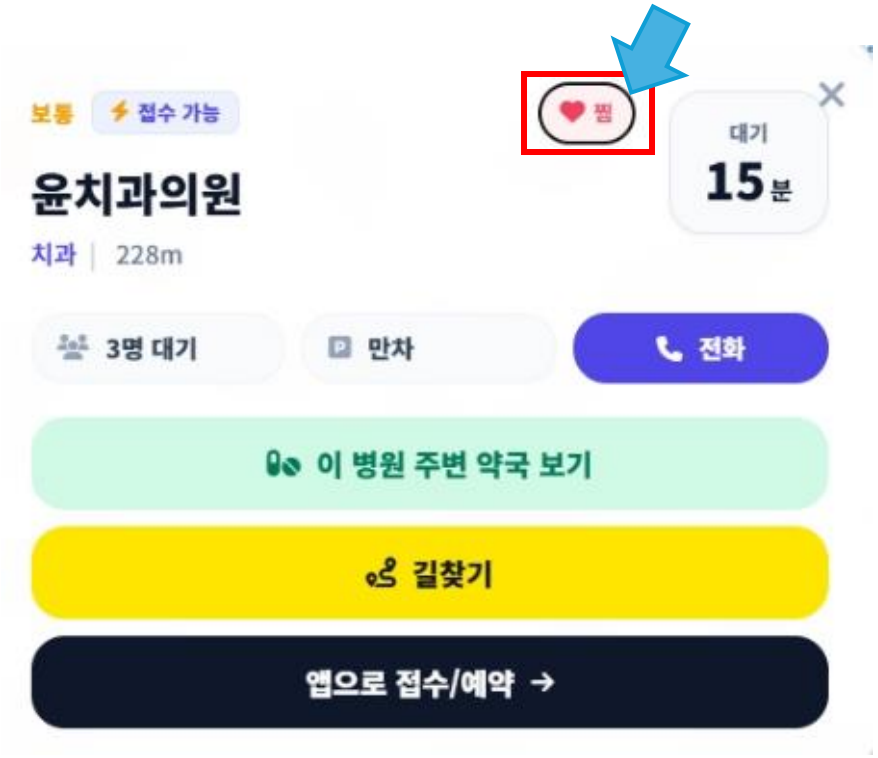

Review
Review
Review
Review
Review DTO
ReviewRes
• Manages auditing dates
• Auth Info Binding
• Auth Formatting
@ManyToOne
User
@ManyToOne)
• BaseEntity
User
• BaseEntity

리뷰 작성 기능


Lazy Loading
@ManyToOne(fetch = LAZY)
• Review.builder().idx(reviewIdx).build()
Security
Flexibility
Efficiency

1

즐겨찾기 기능 (Bookmark)



찜 버튼 클릭 시

즐겨찾기 목록에 추가

2

보안 결함 및 데이터 오염 방어

클라이언트 측 식별자(userId) 전송 시 데이터 조작 가능성 상존

Before (취약)

식별자 직접 수신

```
{
  "userId": 99,     // 조작 가능
  "KakaoPlaceId": 5
}
```

After (안전)

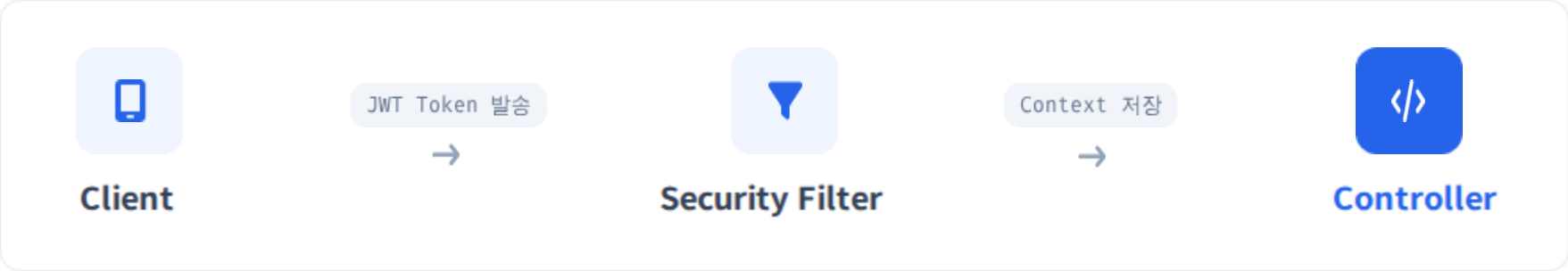
서버 측 토큰 추출

```
// Authorization: Bearer eyJhbGc
iOiJIUzI1Ni...
{
  "KakaoPlaceId": 5
}
```

3

인증 정보의 안전한 주입

@AuthenticationPrincipal을 활용한 토큰 기반 사용자 식별



BookmarkController.java

```
@PostMapping("/register")   mihub02
public ResponseEntity<BookmarkDto.Res> register(
        @AuthenticationPrincipal @NotNull AuthUserDetails userDetails,
        @RequestBody BookmarkDto.Reg dto) {
    BookmarkDto.Res response = bookmarkService.register(userDetails.toEntity(), dto);
    return ResponseEntity.ok(response);
}
```

4

성능 개선

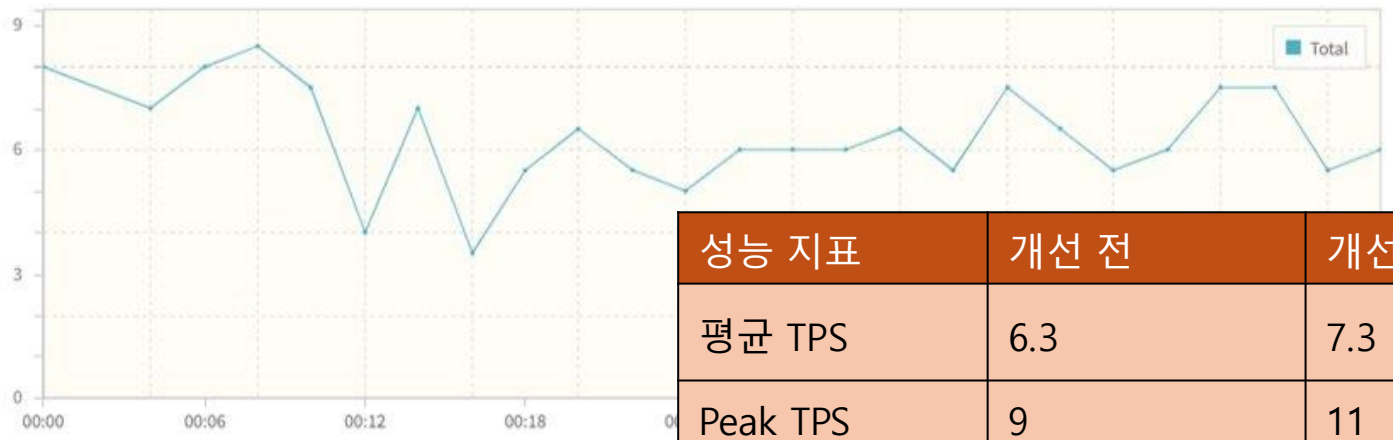
@Cacheable, @Transactional

BookmarkService.java

```
@Transactional(readOnly = true)  1 usage  mihub02
@Cacheable(value = "bookmark", key = "#idx")
public BookmarkDto.Res read(Long idx) {
    Bookmark bookmark = bookmarkRepository.findById(idx).orElseThrow(
            () -> new RuntimeException("북마크 X")
    );
    return BookmarkDto.Res.from(bookmark);
}
```

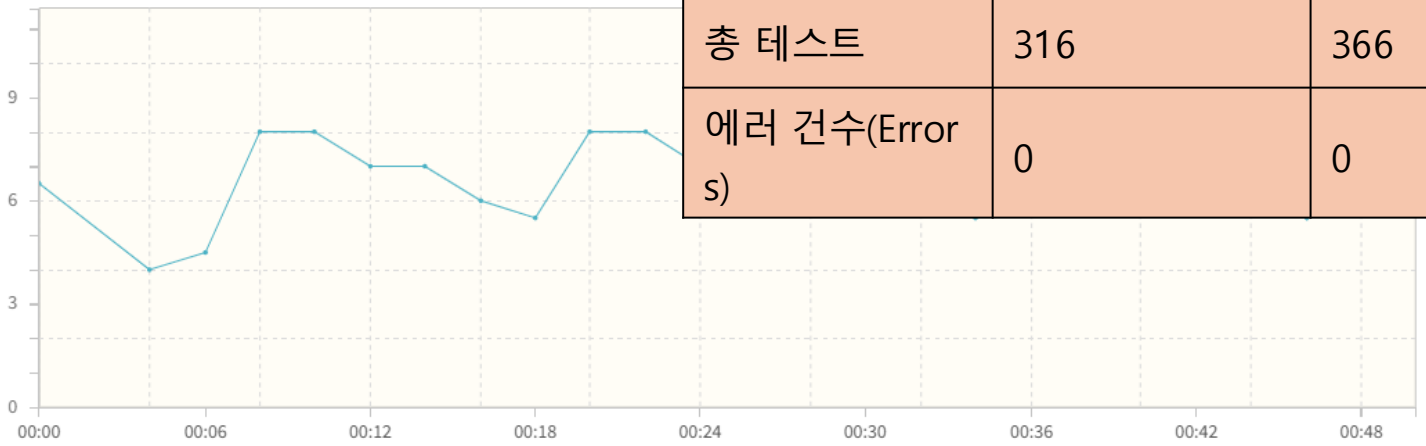
TPS	6.3
Peak TPS	9
Mean Test Time	154.50 ms
Executed Tests	316
Successful Tests	316
Errors	0

Performance Report



TPS	7.3
Peak TPS	11
Mean Test Time	133.60 ms
Executed Tests	366
Successful Tests	366
Errors	0

Performance Report

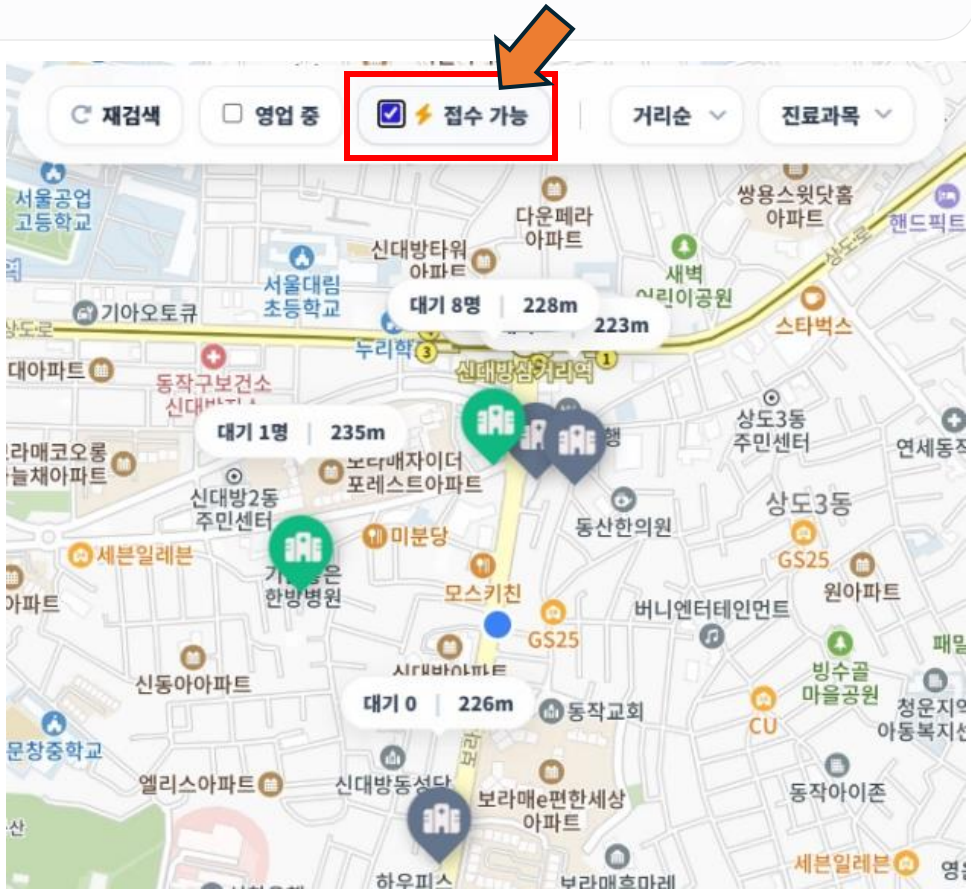


성능 지표	개선 전	개선 후	개선율
평균 TPS	6.3	7.3	약 15.9% 향상
Peak TPS	9	11	약 22.2% 향상
평균 응답 시간	154.5 ms	133.6 ms	약 13.5% 단축
총 테스트	316	366	50건 추가 처리
에러 건수(Errors)	0	0	-

1

제휴 병원 필터링

재검색
영업 중
접수 가능
거리순
진료과목
대기 8명 | 228m
223m
대기 1명 | 235m
대기 0 | 226m
서울공업 고등학교
다운페라 아파트
쌍용스윗닷홈 아파트
신대방타워 아파트
서울대림 초등학교
기아오토큐
스타벅스
동작구보건소
상도3동 주민센터
연세동작
보라매코오롱 하늘채아파트
신대방2동 주민센터
동산한의원
상도3동
GS25
원아파트
세븐일레븐
모스키친
버니엔터테인먼트
신동아아파트
동작교회
빙수골 마을공원
CU
문창중학교
엘리스아파트
보라매e편한세상 아파트
동작아이존
하우피스
보라매휴마레


재검색
영업 중
접수 가능
거리순
진료과목
대기 8명 | 228m
223m
대기 1명 | 235m
대기 0 | 226m
서울공업 고등학교
다운페라 아파트
쌍용스윗닷홈 아파트
신대방타워 아파트
서울대림 초등학교
기아오토큐
스타벅스
동작구보건소
상도3동 주민센터
신대방2동 주민센터
동산한의원
상도3동
GS25
원아파트
세븐일레븐
미분당
모스키친
버니엔터테인먼트
신동아아파트
동작교회
빙수골 마을공원
CU
문창중학교
엘리스아파트
보라매e편한세상 아파트
동작아이존
하우피스
보라매휴마레

2

직관적인 검색바 '접수 가능' 태그 필터링 도입

프론트엔드 단독 필터링으로 즉각적인 제휴 병원 식별

MainMap.vue (Frontend)

```
const applyFilters = () => { let result =  [...rawData.value];
// '접수 가능' 태그 클릭 시, 제휴 병원만 즉시 화면에 남김
if (filterReservableOnly.value) {
result = result.filter(h => h.isReservable === true);}
// 서버 요청 없이 즉각적인 마커 재배치 완료
updateMarkers(result); };
```

즉각적인 목적지 도출
검색바의 **[접수 가능]** 태그 클릭 시, 클라이언트 단에서 isReservable 속성으로 즉시 마커를 걸러냅니다.

Webpush & Serviceworker

사용자가 웹 페이지를 열어두기만 하면 병원 대기 순번을 놓치지 않도록
Service Worker와 VAPID 프로토콜을 활용한 실시간 웹 푸시 시스템을 구축했습니다.

Real-time Notification with Web Push

01. 권한 획득 및 서비스 워커 등록 → 02. 구독 정보 생성 → 03. Spring Server에 구독 정보 저장

04. 백엔드 이벤트 발생 및 푸시 발송 → 05. 서비스 워커의 백그라운드 수신

기대 효과

JUST WEB

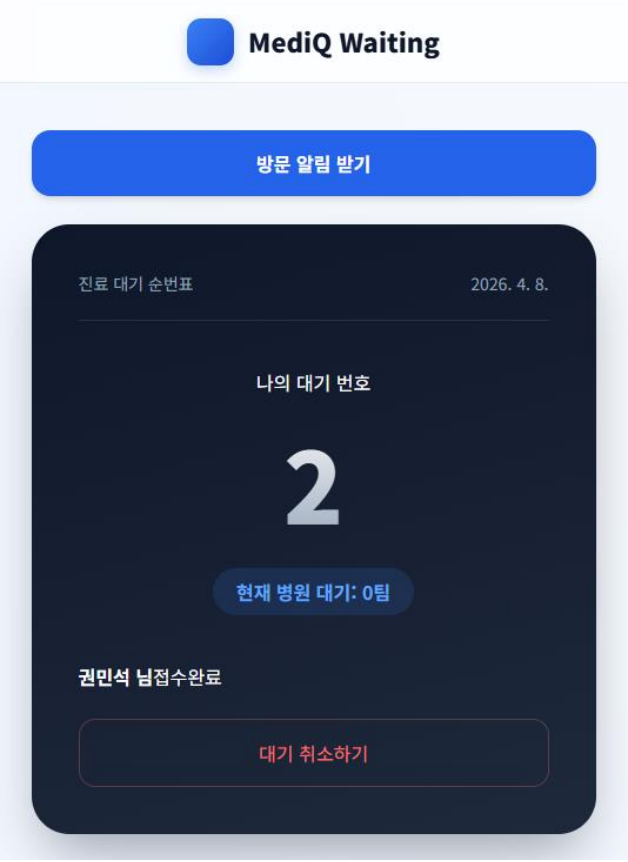
별도의 앱 설치 없이도 모바일 앱 수준의 푸시 알림을 제공하여 환자의 서비스 이탈을 방지합니다.

효율적인 대기 관리

병원 관계자가 대기열을 호출할 때 환자에게 즉각 전달됨으로써 병원 운영의 효율성을 극대화합니다.

대기열 등록 화면

MediQ Waiting
제휴 병원 검색
그린필치과
MEDIQ PARTNER
그린필치과
접수 정보 확인
방문 병원
그린필치과
신청인
권민석
현재 대기 팀
0팀
실시간 대기 신청하기


MediQ Waiting
방문 알림 받기
진료 대기 순번표
2026. 4. 8.
나의 대기 번호
2
현재 병원 대기: 0팀
권민석 님접수완료
대기 취소하기

성능개선 - WaitingController

```java
@GetMapping("/queue/list/{hospitalIdx}")
public ResponseEntity findQueueListByHospitalIdx(
        @PathVariable("hospitalIdx") Long hospitalIdx
) {
    List<WaitingDto.ListRes> waitingList = waitingService.findListByHospitalId(hospitalIdx);
    return ResponseEntity.ok(waitingList);
}
```

```java
@GetMapping("/queue/list/{hospitalIdx}")
public ResponseEntity<Page<WaitingDto.ListRes>> findQueueListByHospitalIdx(
        @PathVariable("hospitalIdx") Long hospitalIdx,
        @RequestParam(value = "page", defaultValue = "0") int page,
        @RequestParam(value = "size", defaultValue = "10") int size
) {
    // Service 호출 시 사용자가 요청한 페이지 정보 전달
    Page<WaitingDto.ListRes> waitingList = waitingService.findListByHospitalIdx(hospitalIdx, page, size);
    return ResponseEntity.ok(waitingList);
}
```

성능개선 - WaitingService

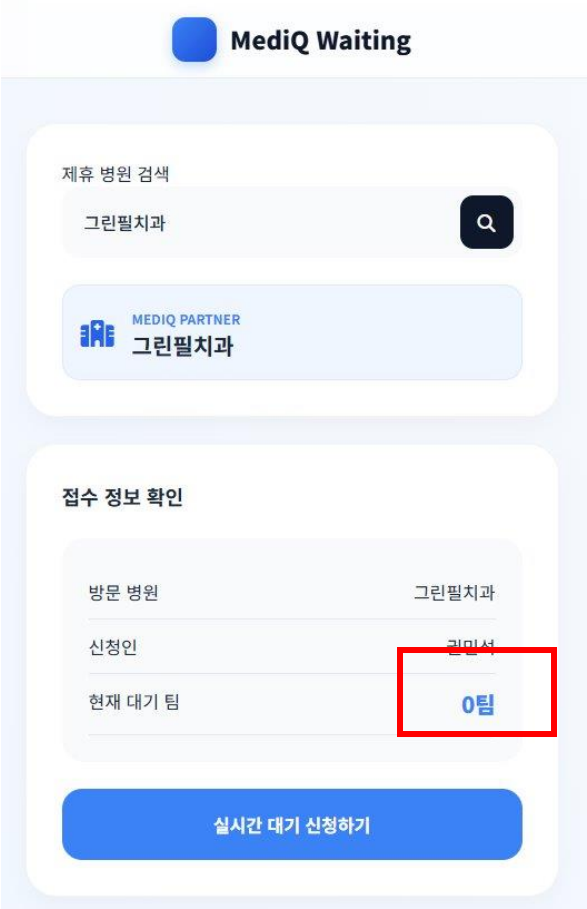
```java
public List<WaitingDto.ListRes> findListByHospitalId(Long hospitalIdx) {  1 usage
    List<Waiting> waitingEntityList = waitingRepository.findAllByHospitalIdx(hospitalIdx);
    List<WaitingDto.ListRes> listResDtoList = new ArrayList<>();
    for (Waiting waiting : waitingEntityList) {
        listResDtoList.add(WaitingDto.ListRes.from(waiting));
    }
    return listResDtoList;
}
```

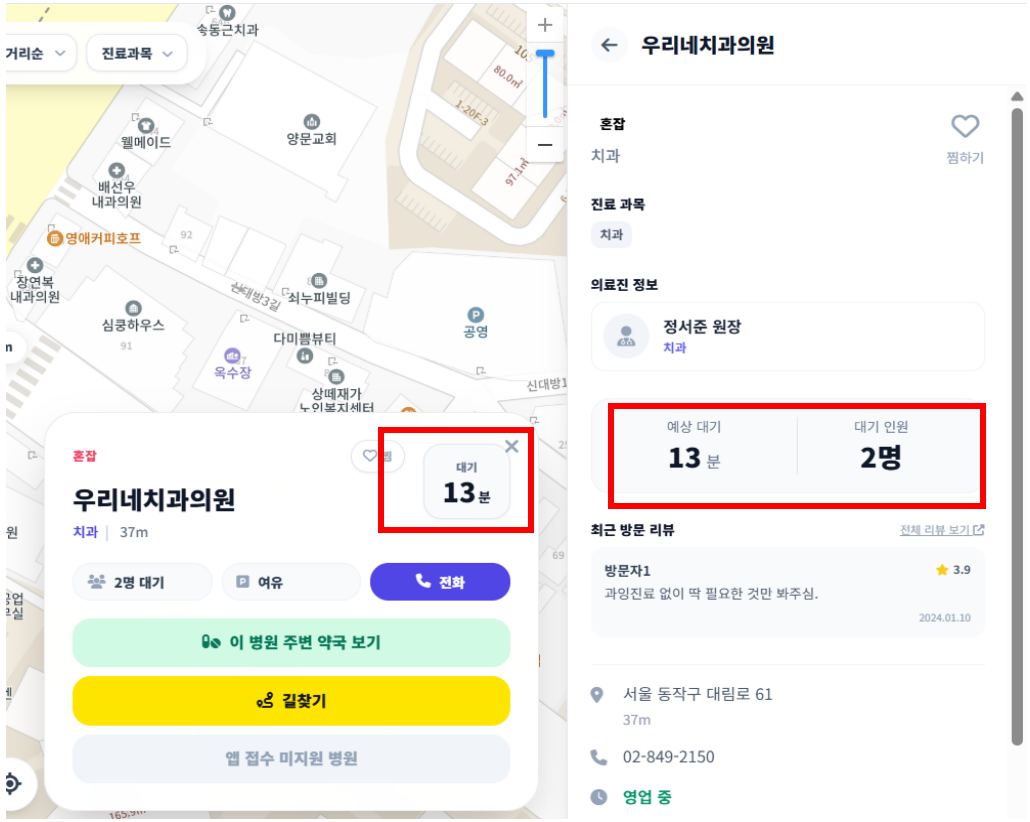
```java
public Page<WaitingDto.ListRes> findListByHospitalIdx(Long hospitalIdx, int page, int size) {
    Pageable pageable = PageRequest.of(page, size, Sort.by( ...properties: "idx").ascending());

    Page<Waiting> waitingPage = waitingRepository.findAllByHospitalIdx(hospitalIdx, pageable);

    List<Waiting> content = waitingPage.getContent();
    List<WaitingDto.ListRes> listResDtoList = new ArrayList<>(content.size());

    for (int i = 0; i < content.size(); i++) {
        listResDtoList.add(WaitingDto.ListRes.from(content.get(i)));
    }

    return new PageImpl<>(listResDtoList, pageable, waitingPage.getTotalElements());
}
```

성능개선 - Ngrinder

성능 지표	개선 전	개선 후	개선율
평균 TPS	6.1	7.7	약 26% 향상
Peak TPS	10	10	동일
평균 응답 시간	162.82 ms	127.04 ms	약 21.9% 단축
총 테스트	304	384	80건 추가 처리
에러 건수(Errors)	0	0	-

Retrospective & Improvement





권 민 석

Queue
Notification

https://github.com/rimin0650

김 미 정

Bookmark
Search

https://github.com/mihub02

노 승 찬

Add Family
Review

https://github.com/seungchan-0629

이 한 별

User
Payment
My page

https://github.com/sole0714